

Rapport — Data Lake IoT industriel

Auteur : Alexis Halbot-Schoonaert

1. Introduction & contexte

Ce projet répond à une mission fictive confiée par la DSI d'un équipementier automobile, accompagné par l'ESN IndustrIA : concevoir et déployer un **data lake** pour centraliser, documenter, sécuriser et gouverner les données de **5 lignes de production** instrumentées de capteurs (température, pression, temps de fonctionnement). Ces données, aujourd'hui stockées « en vrac », doivent constituer la **fondation d'un futur projet de maintenance prédictive** (détection d'anomalies, anticipation des pannes) : le lac ne fait pas la prédiction, mais il rend les données **exploitables, traçables et fiables** pour un tel usage.

La stack est **imposée** par le brief : **MinIO** (stockage objet compatible S3), **Apache Airflow** (orchestration), **OpenMetadata** (catalogue de métadonnées), **Python/boto3**, **Docker Compose** et **Git**. Aucune technologie hors périmètre n'a été introduite.

Toute la stack est décrite dans un unique `docker compose`, relançable à l'identique par un tiers en une seule commande — un **environnement opérationnel et reproductible** qui conditionne toute la chaîne (ingestion, orchestration, catalogue, gouvernance) et garantit que les résultats sont vérifiables. Le journal de réalisation, dont ce rapport est la consolidation, retrace les étapes par **date réelle** ; la procédure d'installation et d'exécution complète figure dans le README.

Le projet couvre **quatre volets**, détaillés en §3 : l'**architecture** (C18), l'**intégration** (C19), le **catalogue et le cycle de vie** (C20), la **sécurité et la gouvernance** (C21).

2. Architecture du data lake

Le lac est organisé en **quatre couches**, chacune avec une responsabilité unique, qui matérialisent le raffinement progressif de la donnée depuis le fichier brut jusqu'à la table prête à l'analyse :

```
data/ (CSV source)
| ingestion (byte-identique + MD5)
▼
raw/      production_lines/lineX/year=/month=/      (brut, immuable)
| harmonisation (Parquet, schéma unifié)
▼
staging/  production_lines/lineX/year=/month=/day=/  (nettoyé)
| consolidation (table unifiée)
▼
curated/  production_lines/line=lineX/year=/month=/day=/ (prêt à l'analyse)
| archiver (DAG) —————> archive/ (rétention, expiration ILM)
```

Séparer ces couches sert trois objectifs : garantir la **reproductibilité** (le brut n'est jamais retouché, tout est rejouable), améliorer la qualité **progressivement**, et **tracer** le cheminement de la donnée.

Deux choix structurants sous-tendent ce modèle. D'abord, un **partitionnement à deux granularités** : `raw` est partitionné au **mois** (`lineX/year=/month=/`), ce qui permet de déposer les fichiers source *tels quels* — un fichier couvrant exactement un mois — et d'en vérifier l'intégrité ; `staging` et `curated` sont partitionnés au **jour** (`.../day=DD/`), granularité fine alignée sur un traitement **au fil de l'eau** (une journée à la fois) et sur des requêtes journalières, qui permet aussi un **élagage** précis à la lecture.

Ensuite, un choix de **formats** : le **CSV** est conservé en `raw` (fidélité absolue à la source), tandis que l'aval passe au **Parquet** — typé, compressé, colonnaire — bien plus adapté à l'analytique.

MinIO s'impose naturellement : stockage objet **compatible S3**, il se pilote avec la bibliothèque **boto3** standard. Le modèle détaillé, les justifications de volumétrie/fréquence et le schéma annoté sont dans docs/architecture.md.

3. Réalisations par compétence

3.1 C18 — Analyse des données & architecture

L'analyse a précédé toute décision technique. Les 5 fichiers ont été récupérés depuis Zenodo via son API REST, avec **vérification d'intégrité MD5**, puis explorés (volumétrie, schémas, types, distribution du label, couverture temporelle). Constats : **30 000 enregistrements** au total, **un relevé par minute**, de janvier à mai 2025, avec **environ 1 % d'anomalies**. LineA compte à elle seule **10 000 enregistrements**, traités **en chunks** pour simuler un flux réel.

L'exploration s'appuie sur des **fonctions réutilisables** (regroupées dans le package `datalake/`), ce qui la rend rejouable. Deux propriétés vérifiées structurent la suite : le **timestamp** est **régulier et continu** (une mesure par minute, sans trou ni doublon) et chaque fichier source couvre **exactement un mois** — d'où l'horodatage fiable et le partitionnement mensuel de `raw`. Enfin, le **déséquilibre du label** (~1 % d'anomalies) est déterminant pour la finalité : la détection d'anomalies porte précisément sur ces cas rares, que le lac doit **préserver fidèlement**, sans les diluer ni les perdre à l'harmonisation.

La difficulté centrale du brief est l'**hétérogénéité des schémas** entre lignes, que le catalogue rend visible (colonnes `raw`, hors partitions) :

Concept	lineA	lineB	lineC	lineD	lineE
timestamp	timestamp	timestamp	timestamp	timestamp	timestamp
temperature	Temperature	temperature	Temperature	temperature	Temperature
pressure	pressure	pressure	pressure	Pressure	pressure
elapsed_time	elapsed_time	Elapsed_time	absent	absent	absent
label	label	label	label	label	label

Trois écarts sont à traiter : la **casse** des colonnes (`Temperature/temperature`, `Pressure` sur lineD), la casse d'`Elapsed_time` (lineB), et surtout la **présence/absence d'elapsed_time** (présent sur A et B, absent de C/D/E). L'analyse a également relevé un **écart documentation vs données** — LineE annoncée « 0 % d'anomalies » en contient en réalité 0,5 % : *la donnée fait foi*, l'écart est tracé comme avertissement qualité.

Ces constats ont directement nourri les **décisions d'architecture** : un **schéma cible unifié** (colonnes en minuscules, `timestamp` en ISO 8601, `elapsed_time` en NULL si absent), une **clé naturelle (line, timestamp)** garantissant l'idempotence, et le partitionnement décrit plus haut. L'exploration est reproductible via notebooks/exploration_jour1.ipynb et le schéma annoté dans docs/architecture.md.

3.2 C19 — Intégration

Toute la stack est **reproductible** via Docker Compose : buckets, comptes, bases et orchestrateurs se créent au démarrage. L'**ingestion** dépose les CSV de `data/` vers `raw/` de façon **byte-identique**, contrôle l'**intégrité par MD5** (comparaison à l'ETag renvoyé par MinIO) et reste **idempotente** : réimporter un fichier inchangé n'a aucun effet, tandis qu'un fichier modifié déclenche le réimport et l'invalidation des couches dérivées.

Le pipeline est orchestré par **3 DAGs Airflow** — `ingestion_raw` (`data` → `raw`, déclenché manuellement), `harmonisation_staging` (`raw` → `staging`) et `consolidation_curated` (`staging` → `curated`) — conçus comme

des **coquilles fines** : ils ne contiennent **aucune logique métier** et se bornent à appeler les fonctions du package `datalake/` depuis des `PythonOperator`. Le même code s'exécute donc à l'identique dans Airflow et dans un conteneur de développement prévue dans Docker Compose.

La couche **staging harmonise** les schémas hétérogènes vers le schéma cible (casse, `timestamp` ISO 8601, `elapsed_time` nullable, écriture Parquet, dédoublonnage) ; la couche **curated consolide** en une table unifiée où la ligne devient une partition `line=`. Le tout a été validé de bout en bout sur MinIO réel (**23 partitions** produites en **staging** puis en **curated**).

Un point clé de robustesse est la **cascade d'invalidation** : réimporter une (ligne, mois) dans **raw purge automatiquement** les partitions correspondantes de **staging** et de **curated**, que les filigranes reconstruisent ensuite. Chaque partition est ainsi **idempotente** (la rejouer réécrit son contenu à l'identique) et le pipeline reste cohérent même après une correction de la source. La reproduction de bout en bout par un tiers suit un enchaînement documenté — téléchargement des sources (avec MD5), ingestion vers **raw**, puis exécution des DAGs — **relançable sans effet de bord**.

La qualité du code repose sur un développement en **TDD pur Python** (un faux client S3 en mémoire, qui teste la logique sans dépendre de MinIO), l'usage de **Polars** plutôt que de pandas (Arrow-natif, avec un `null` distinct du `NaN`), et un outillage homogène **ruff + pytest + typage strict**. Les DAGs et la procédure d'intégration sont documentés dans `dags/` et le README.

3.3 C20 — Catalogue & cycle de vie

Le **catalogue OpenMetadata** documente les données et rend leur circulation traçable. L'intégration est faite en **config-as-code** : un **service de stockage S3** connecté à MinIO, décrit par des fichiers YAML versionnés et lancé par la CLI `metadata` — donc **reproductible sans clic manuel**. Il en résulte les 4 buckets et **16 conteneurs structurés** (un par ligne et par couche).

Les **5 fiches raw** (une par ligne) sont enrichies : description avec la **source Zenodo**, la **fréquence** (1 relevé/minute) et la sémantique de **label**, **propriétaire** (équipe *Responsable maintenance*), et **colonnes documentées** avec unités (température °C, pression bar). Le catalogue **rend visible l'hétérogénéité** des sources en **raw** et son harmonisation en **staging/curated**.

Les **4 DAGs** sont catalogués comme *Pipelines* et le **lignage** entre conteneurs est déduit d'annotations **inlets/outlets** (métadonnées pures, sans effet d'exécution) :

```
raw.lineX -(harmonisation_staging)-> staging.lineX -(consolidation_curated)-> curated.line=lineX
raw.lineX -(archivage)-> archive.lineX
```

Le catalogue est **observationnel** : il ne modifie ni la logique des DAGs ni les buckets. La procédure complète est dans `init-scripts/openmetadata/README.md` et les **captures d'écran** (service, fiches, lignage) sont rassemblées dans `docs/captures-openmetadata/`.

Ce catalogue apporte une **traçabilité** concrète : on visualise d'où vient chaque conteneur, par quel DAG il est produit et comment l'hétérogénéité de **raw** est résorbée en aval — une base essentielle pour la **découverte** des données et pour la **confiance** qu'on peut leur accorder.

Le **cycle de vie** organise le vieillissement des données en deux temps. L'**archivage** est confié à un **DAG** dédié : pour chaque (ligne, mois) dont la **date des données** dépasse le seuil, il copie l'objet de **raw** vers **archive/** (chemin en miroir) puis **purge** les partitions correspondantes de **staging** et **curated**. La **suppression**, elle, relève d'une **règle ILM** d'expiration posée sur **archive/** (730 jours, fondée sur l'âge de l'objet).

Ce partage n'est pas arbitraire : l'ILM de MinIO ne sait faire que l'**expiration** ou la **transition vers un tier distant** — il n'existe **aucun transfert local de bucket à bucket** —, si bien que « déplacer

raw/ vers archive/ en local » **exige un DAG**, l'ILM restant employé pour l'opération qu'il sait faire, la suppression. La politique complète figure dans docs/gouvernance-cycle-de-vie.md.

3.4 C21 — Sécurité & gouvernance

L'accès est cloisonné par le principe du **moindre privilège**. Trois comptes de service sont créés avec des **polices IAM personnalisées, restreintes par bucket** (là où les polices intégrées de MinIO porteraient sur *tous* les buckets) :

Rôle (compte)	raw	staging	curated	archive
data-analyst	—	—	lecture	—
data-engineer	lecture/écriture	lecture/écriture	lecture/écriture	lecture/écriture
datalake-admin	tous droits	tous droits	tous droits	tous droits

L'analyste ne voit que les données prêtes à l'analyse (**curated**, en lecture seule) ; le data-engineer couvre les quatre couches — **archive** compris, car il **porte le cycle de vie** (archivage et réintégration) ; l'administrateur gère l'infrastructure. Le compte root n'est pas utilisé au quotidien.

Le **chiffrement au repos SSE-S3** est activé sur les 4 buckets via le **KMS intégré** de MinIO : le serveur chiffre et déchiffre de façon **transparente** pour les clients (boto3, mc, DuckDB), sans aucune gestion de clé côté application. Le chiffrement a été vérifié sur un objet témoin (Encryption: SSE-S3). La **matrice des droits** ci-dessus, les conditions d'accès et les **responsabilités par rôle** sont formalisées dans docs/gouvernance-acces-securite.md.

Au-delà des droits techniques, la **politique de gouvernance** écrite précise les **conditions d'accès** (comptes de service dédiés, secrets hors dépôt, réseau interne) et les **responsabilités par rôle** : l'analyste consulte, le data-engineer construit et maintient le pipeline et le cycle de vie, l'administrateur gère l'infrastructure et les clés. Cette formalisation rend la gouvernance **auditable et transmissible**, indépendamment des personnes.

4. Choix techniques & difficultés

Plusieurs décisions transverses ont structuré le projet. Le **catalogue en config-as-code** (plutôt que la configuration à la souris dans l'UI) rend l'ingestion **versionnée et reproductible** par un tiers. La **mutualisation de Postgres** (trois bases isolées dans un seul conteneur) est un gain net sans couplage fonctionnel. Les tâches d'amorçage — création des buckets, initialisation d'Airflow, migration d'OpenMetadata — sont des **jobs one-shot** qui s'exécutent puis s'arrêtent, et le **périmètre reste minimal** (uniquement les briques de la stack imposée, sans service annexe).

Un **conteneur de développement** dédié exécute le code métier **dans le réseau Docker** de la stack : les noms d'hôte (minio:9000, postgres...) y résolvent exactement comme pour Airflow, de sorte que le **même code** s'exécute sans adaptation, en débogage comme en production. Il **facilite l'exécution des scripts** (python -m datalake... lancés directement contre les services réels) et offre une **intégration native avec l'IDE et le débogage pas-à-pas** (points d'arrêt sur du code qui parle au vrai MinIO), sans exposer de ports ni dupliquer la configuration.

Dans le même esprit d'intégration hôte/conteneur, les **UID/GID** des conteneurs sont **alignés sur ceux de l'utilisateur hôte** (id -u/id -g) : les fichiers écrits dans les **volumes partagés** (bind mounts de dags/, datalake/) appartiennent alors à l'utilisateur, ce qui **évite les problèmes de droits** classiques (fichiers créés en root par le conteneur) entre le système de fichiers hôte et les conteneurs.

Enfin, l'exploration du **contenu** des données en SQL a été démontrée avec **DuckDB** (lecture directe des Parquet sur MinIO, en ligne de commande et via DBeaver), effectuée sous le compte **data-analyst** — ce qui **illustre concrètement la gouvernance par bucket**, les autres couches lui restant inaccessibles. Comme chaque fichier Parquet correspond à une partition (**ligne, jour**), DuckDB exploite en outre les **statistiques de partition** pour n'ouvrir que les fichiers pertinents, ce qui accélère les requêtes filtrées.

Les difficultés rencontrées ont été de trois ordres : l'**hétérogénéité des schémas** (résolue par le schéma cible unifié), la **forte sensibilité aux versions d'OpenMetadata** (serveur, ingestion et Elasticsearch devant rester cohérents), et les **limites de l'ILM objet** (pas de copie locale bucket-à-bucket), qui ont conduit au partage DAG / ILM décrit plus haut.

Le filigrane auto-réparant — un choix de conception assumé. Le pipeline au fil de l'eau est piloté par l'**état des données**, non par la date d'exécution Airflow : à chaque exécution, un **filigrane** désigne le **plus ancien jour présent en amont mais absent en aval**, et cette journée-là est traitée (une par exécution). Ce mécanisme rend la **réintégration transparente** : recopier un mois de **archive/** vers **raw/** déclenche la **purge en cascade** des couches dérivées de ce mois, le filigrane **recule** en conséquence, et **staging puis curated se recréent seuls**, sans aucune action manuelle — c'est bien la combinaison **filigrane + purge en cascade** qui produit l'auto-réparation, le filigrane seul n'y suffirait pas.

Ce mécanisme a toutefois un revers : balayer le backlog pour n'en traiter qu'un jour par exécution convient à la **simulation d'un flux** et au **rattrapage**, mais **pas à un véritable flux de production**, où l'on indexerait plutôt sur la **date d'exécution / l'intervalle de données** (traitement *en avant*, du jour courant) avec un mécanisme de **backfill explicite** pour corriger le passé. Autrement dit, cette logique « recalculer le plus ancien trou » n'a de sens que **parce que l'on réimporte des jours passés** (démonstration et réintégration) : c'est un dispositif pédagogique adapté au contexte du projet, à distinguer d'une pratique de production.

Les **notions abordées** au fil du projet sont : architecture en couches et partitionnement d'un data lake ; **idempotence** et clé naturelle ; **intégrité** par hash MD5 ; gestion des valeurs manquantes (le **null** natif de Polars, distinct du **NaN**) ; **déséquilibre de classes**, enjeu clé de la détection d'anomalies ; orchestration Airflow et séparation logique/orchestration ; **filigrane** comme état dérivé des données ; gouvernance des métadonnées et **lignage** ; modèle **IAM/RBAC** et **chiffrement au repos**.

5. Auto-évaluation par compétence

Récapitulatif de ce qui a été réalisé pour chaque compétence.

- **C18 — Architecture & analyse** : les 5 lignes ont été analysées **avant** toute décision technique (volumétrie, schémas, hétérogénéités) ; l'architecture en couches a été modélisée au regard de la volumétrie et de la fréquence ; un schéma technique annoté a été produit.
- **C19 — Intégration** : stack reproductible via Docker Compose ; ingestion vers **raw/** avec **vérification MD5** et **idempotence** ; **3 DAGs** (ingestion → harmonisation → consolidation) en fil-de-l'eau ; LineA traitée par jour (chunks) ; procédure d'intégration documentée dans le README.
- **C20 — Catalogue & cycle de vie** : **catalogue OpenMetadata** config-as-code (service S3 → MinIO, 5 fiches **raw** avec colonnes, propriétaire, source, fréquence), 4 pipelines et **lignage raw→staging→curated + raw→archive**, catalogue **non-invasif** ; **cycle de vie** : archivage **raw→archive** par DAG (l'ILM ne copiant pas localement) et **expiration** par règle ILM, réintégration par filigrane.
- **C21 — Sécurité & gouvernance** : **3 comptes** aux droits différenciés par bucket, **chiffrement SSE-S3** au repos sur les 4 buckets (KMS intégré), **matrice des droits + politique de gouvernance** écrite.
- **Décisions au-delà de la lettre de l'énoncé** (chacune motivée dans les sections précédentes) : 3e DAG de consolidation, cadence minute simulant un flux, **filigrane auto-réparant**, **conteneur**

de développement intégré au réseau Docker (débogage IDE contre les services réels), exploration SQL via **DuckDB**.

6. Conclusion

Le projet livre un **data lake opérationnel, reproductible, catalogué et sécurisé**. Les données des 5 lignes de production sont ingérées avec contrôle d'intégrité, harmonisées malgré leurs schémas hétérogènes, consolidées en une table prête à l'analyse, documentées et tracées dans OpenMetadata, soumises à un cycle de vie, et protégées par un cloisonnement des accès et un chiffrement au repos.

L'ensemble est **relançable par un tiers** via Docker Compose et entièrement versionné. Le lac constitue ainsi la **fondation** sur laquelle pourra s'appuyer le futur projet de maintenance prédictive.

7. Annexes

Documentation du projet (également indexée dans le README) :

- README.md — **installation et utilisation** : reproduction de l'environnement, exécution du code, sommaire + index de la documentation.
- docs/architecture.md — **architecture** en couches, partitionnement, schéma annoté, contrat d'implémentation (C18).
- docs/gouvernance-cycle-de-vie.md — politique de **cycle de vie** : archivage **raw→archive** par DAG + expiration ILM (C20).
- docs/gouvernance-acces-securite.md — **matrice des droits** + politique de gouvernance & chiffrement SSE-S3 (C21).
- init-scripts/openmetadata/README.md — procédure d'ingestion du **catalogue OpenMetadata** (config-as-code) : conteneurs, fiches, lignage.
- docs/captures-openmetadata/README.md — **captures d'écran** du catalogue (C20).
- journal.md — **journal de réalisation** au fil de l'eau (activités, notions, choix par date) dont ce rapport est la consolidation.

Autres ressources :

- Exploration des données (Jour 1) : notebooks/exploration_jour1.ipynb.
- Dépôt Git organisé (structure du dépôt) : voir le README.